iOS SDK Introduction

WeDoBooks SDK Integrator Guide

Setup and Authentication

Goal

Initialize SDK safely, choose backend mode explicitly, and authenticate users before book operations.

Integrator Steps

1. Add SPM dependency from the SDK umbrella repo.
2. Provide a `FirebaseAdapterFactoryType` implementation from FirebaseProvider.
3. Call `WeDoBooksFacade.shared.setup(...)` once per app launch on main actor.
4. Fetch a custom token from your backend and sign in using `userOperations.signIn(with:)`.

Setup Example (Default Firebase App)

```
import WeDoBooksSDK
import FirebaseProvider

@MainActor
func configureSDK() throws {
    try WeDoBooksFacade.shared.setup(
        readerKey: "<reader-key>",
        readerSecret: "<reader-secret>",
        mode: .streaming,
        firebaseAdapterFactory: FirebaseAdapterFactory(),
        firebaseConfigFilePath: nil
    )
}
```

Setup Example (Named Firebase App)

Use a custom plist when the SDK should use a non-default Firebase app.

```
import WeDoBooksSDK
import FirebaseProvider

enum SDKSetupExampleError: Error {
    case missingFirebaseConfigFile(String)
}

@MainActor
func configureSDKWithNamedFirebasePlist() throws {
    let plistName = "GoogleService-Info-WDBSDK"
     guard let configPath = Bundle.main.path(forResource: plistName, ofType: "plist")
else {
        throw SDKSetupExampleError.missingFirebaseConfigFile(plistName)
    }

    try WeDoBooksFacade.shared.setup(
        readerKey: "<reader-key>",
        readerSecret: "<reader-secret>",
```

```
        mode: .streaming,
        firebaseAdapterFactory: FirebaseAdapterFactory(),
        firebaseConfigFilePath: configPath
    )
}
```

Token Acquisition Guidance

- Production: Host app requests token from your backend. Your backend performs backend-to-backend token exchange with the WeDoBooks token endpoint and returns a short-lived custom token to the host app.
- Testing: You can expose a demo token endpoint pattern (for example `https://example-integrator.test/wedobooks/demo-token`) that returns a token for sandbox testing flows.

Security note: Demo token endpoints are for testing and non-production flows only.

Auth Example (Sign In With Custom Token)

```
func signIn(token: String) async {
    let result = await WeDoBooksFacade.shared.userOperations.signIn(with: token)
    if case .failure(let error) = result {
        print("Sign in failed: \(error)")
    }
}
```

Flow Diagram

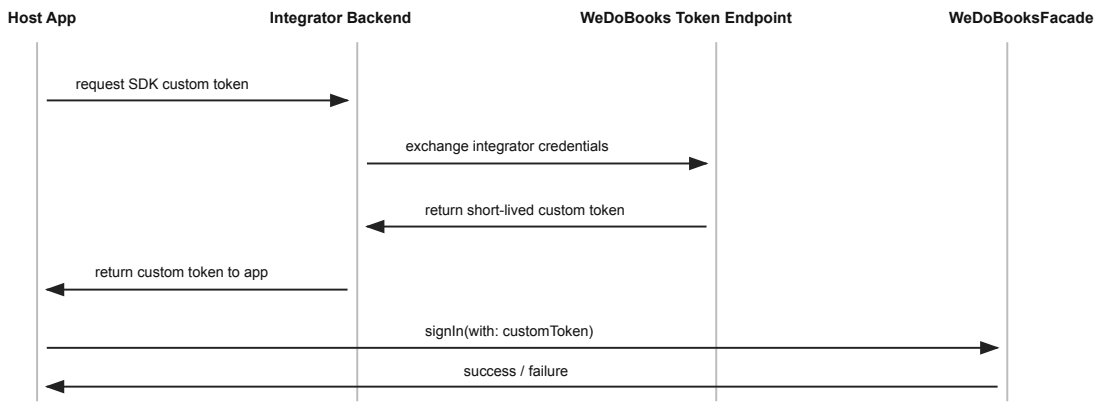

Figure 1: Setup and auth sequence

Optional Demo Token Fetch Example (Testing)

```
import Foundation

struct DemoTokenResponse: Decodable {
    let token: String
}
```

```swift
func fetchDemoTokenForTesting(userId: String) async throws -> String {
    let encodedUser = userId.addingPercentEncoding(
        withAllowedCharacters: .urlQueryAllowed
    ) ?? userId

    let baseURL = "https://example-integrator.test/wedobooks/demo-token"
    let url = URL(string: "\(baseURL)?user_id=\(encodedUser)")!
    let (data, _) = try await URLSession.shared.data(from: url)

    return try JSONDecoder()
        .decode(DemoTokenResponse.self, from: data)
        .token
}
```

Notes

- `setup(...)` is intentionally explicit about backend mode.
- Repeated setup attempts throw `SetupError.setupAlreadyCalled`.
- Most namespaces that interact with user data require a signed-in user.

Cross References

- DocC: GettingStarted.md
- DocC symbol root: `WeDoBooksFacade`

Checkout and Open

Goal

Request/observe user checkouts and present reader/player UI safely.

Recommended Flow

1. Ensure user is signed in.
2. Observe checkouts (`observeCheckouts`) for UI list updates.
3. Request missing checkout (`checkoutBook(with:)`) when needed.
4. Present using `openCheckout(_:presentedBy:customCover:)` on main actor.

Checkout + Open Example

```
func open(isbn: String, presenter: UIViewController) async {
    let result = await WeDoBooksFacade.shared.bookOperations.checkoutBook(with: isbn)
    switch result {
    case .success(let checkout):
        do {
           try WeDoBooksFacade.shared.bookOperations.openCheckout(checkout, presentedBy:
presenter)
        } catch {
            print("Open failed: \(error)")
        }
    case .failure(let error):
        print("Checkout failed: \(error)")
    }
}
```

Notes

- Only one SDK book UI can be open at a time.
- If headless audiobook playback is active, `openCheckout` throws `OpenCheckoutError.aBookIsAlreadyOpen`.
- Ebook open path may show download/progress UI before reader when local file is missing.

Cross References

- DocC: BookOperations.md

Offline and Storage

Goal

Help host apps let users download ebooks and audiobooks, show useful offline state, recover from storage failures, and understand what happens when downloaded material is removed or a user signs out.

Offline support is centered around a `Checkout`. A checkout tells the SDK which material belongs to the signed-in user, and the storage APIs use that checkout to download, inspect, and remove local material.

Offline Model

Downloaded material is scoped to the current signed-in user. The SDK stores downloaded ebook/audio artifacts in app storage, keeps offline bookmarks and related metadata in local preferences, and stores ebook offline keys in the keychain.

Ebooks are always downloaded before they are opened so they are available offline immediately after that and until the material is removed explicitly through the API or the user logs out. Audiobooks can be downloaded through a call to the API. After download has completed successfully then the books can also be opened when offline.

API Quick Reference

Use these APIs to build host UI around the offline model:

- `storageOperations.download(book:)`: Start downloading a checkout for offline use.
- `storageOperations.downloadStatus(book:)`: Render download state and progress in host UI.
- `storageOperations.isBookDownloaded(isbn:)`: Check whether the current user has local material for an ISBN.
- `storageOperations.getAllDownloadedBooks()`: List ISBNs with local offline records for the current user.
- `storageOperations.removeDownload(isbn:)`: Remove one downloaded material item for the current user.
- `storageOperations.removeAll()`: Clear SDK local offline/download storage.
- `storageOperations.onError`: Observe asynchronous download/storage failures.
- `userOperations.signOut()`: Sign out and clear SDK local offline/download state.

`downloadStatus(book:)` is usually the best API for UI because it can distinguish between in-progress downloads, completed downloads, and material that is not downloaded.

Most storage APIs require setup to have completed and a user to be signed in. Calls that cannot even start because setup or sign-in is missing throw synchronously. Download failures that happen after a request has started are reported asynchronously through `storageOperations.onError`.

Download Lifecycle

Start by acquiring a `Checkout`, for example through `bookOperations.checkoutBook(with:)` or `bookOperations.observeCheckouts()`. Pass that checkout to `download(book:)`.

```
import Combine
import WeDoBooksSDK

private var cancellables = Set<AnyCancellable>()

func prepareOfflineAccess(for checkout: Checkout) {
    WeDoBooksFacade.shared
```

```
        .storageOperations
        .onError
        .sink { error in
            print("Offline download failed: \(error)")
        }
        .store(in: &cancellables)

    do {
        try WeDoBooksFacade.shared
            .storageOperations
            .download(book: checkout)
    } catch {
        print("Could not start download: \(error)")
    }
}
```

Use `downloadStatus(book:)` to keep host UI in sync:

```
func refreshOfflineState(for checkout: Checkout) {
    do {
        let status = try WeDoBooksFacade.shared
            .storageOperations
            .downloadStatus(book: checkout)

        switch status {
        case .downloading(let progress):
            print("Downloading \(Int(progress * 100))%")
        case .downloaded:
            print("Available offline")
        case .notDownloaded:
            print("Not downloaded")
        }
    } catch {
        print("Could not read download status: \(error)")
    }
}
```

Status values mean:

- `.downloading(progress:)`: a download is active or initializing. `progress` is a `0.0...1.0` fraction.
- `.downloaded`: local material exists for the current user and checkout.
- `.notDownloaded`: local material is absent, not started, removed, or the latest start attempt failed before a usable download state was established.

To check availability without rendering progress, use:

```
func printDownloadedISBNs() {
    do {
        let isbns = try WeDoBooksFacade.shared
            .storageOperations
            .getAllDownloadedBooks()
        print("Downloaded ISBNs: \(isbns)")
    } catch {
        print("Could not list downloads: \(error)")
    }
}
```

Opening Downloaded Material

Open a checkout the same way whether it has been downloaded or not:

```
@MainActor
func open(_ checkout: Checkout, from presenter: UIViewController) {
    do {
        try WeDoBooksFacade.shared
            .bookOperations
            .openCheckout(checkout, presentedBy: presenter)
    } catch {
        print("Could not open checkout: \(error)")
    }
}
```

For ebooks, offline opening requires both the downloaded file and the SDK's stored offline decryption metadata. If that local information has been removed or is invalid, opening can fail with `OpenCheckoutError.decryptionInformationInvalidOrMissing`. In that case, guide the user to download or open the material again while online.

Disk Space and Download Failures

The public storage API does not expose a dedicated "disk full" error case. If the device runs out of space or the filesystem cannot persist downloaded material, the host app should expect a `StorageOperationError.underlyingError` through `storageOperations.onError` or as a thrown error from a storage operation, depending on where the failure occurs.

A practical recovery path is:

1. Tell the user the download could not be completed.
2. Suggest freeing device storage.
3. Let the user retry the download.
4. Refresh `downloadStatus(book:)` after the failure or retry.

```
func handleStorageError(_ error: StorageOperationError) {
    switch error {
    case .setupNotCompleted:
        print("SDK setup has not completed.")
```

```
    case .noUserSignedIn:
        print("Sign in before using offline storage.")
    case .downloadNotAllowedOnCellular:
        print("Connect to Wi-Fi or allow cellular downloads.")
    case .underlyingError:
        print("The download failed. Free up space and try again.")
    }
}
```

Removing Downloaded Material

Use `removeDownload(isbn:)` to remove a single downloaded item for the current user:

```
func removeOfflineMaterial(isbn: String) {
    do {
        try WeDoBooksFacade.shared
            .storageOperations
            .removeDownload(isbn: isbn)
    } catch {
        print("Could not remove downloaded material: \(error)")
    }
}
```

After a successful removal, the SDK clears the cached download state for that ISBN. A later `downloadStatus(book:)` call should resolve to `.notDownloaded`.

Opening the same material again after removal may require network access so the SDK can stream or download the material again. For ebooks, the removed offline file and decryption metadata cannot be used after removal.

Use `removeAll()` when the host app needs to clear SDK offline storage explicitly:

```
func clearAllSDKOfflineStorage() {
    WeDoBooksFacade.shared
        .storageOperations
        .removeAll()
}
```

`removeAll()` clears downloaded artifacts, bookmarks/offline metadata, ebook offline data, easy-access entries, and cached download-status state.

Logout Behavior

`userOperations.signOut()` signs out the current user and triggers SDK cleanup of local offline storage.

```
func signOut() {
    WeDoBooksFacade.shared
        .userOperations
        .signOut()
```

```
    // SDK local downloaded material and offline metadata are cleared.
}
```

After logout, integrators should expect downloaded material, offline metadata, easy-access data, and cached download-status state to be cleared. A newly signed-in user must establish their own checkout and download state.

Implementation Notes

- Download failures can be synchronous (`throw`) or asynchronous (`onError`).
- Downloads and local records are user-scoped.
- Status is exposed as `StorageDownloadStatus`; progress is available while downloading.

Cross References

- DocC: StorageOperations.md
- DocC: BookOperations.md
- DocC: UserOperations.md

Customization

Goal

Adjust SDK behavior and visual integration without forking SDK internals.

Available Areas

- `configuration`: feature toggles and persisted behavior flags
- `styling`: light/dark theme override
- `localization`: language selection and string overrides
- `images`: icon and reader image overrides
- `events`: host-side reactions to SDK UI actions

Configuration Example

```
let facade = WeDoBooksFacade.shared
facade.configuration.showFinishEbookButton = true
facade.configuration.showFinishAudiobookButton = true
facade.configuration.showAboutAudioBookButton = true
```

Localization Override Example

```
typealias LocalizationOverrides = [
    WeDoBooksFacade.Localization.LocalizationKeys: [
        WeDoBooksFacade.Localization.Language: String
    ]
]

let overrides: LocalizationOverrides = [
    .buttonSave: [.english: "Save"],
    .playerPlaybackRateSpeed: [.english: "Speed"]
]

WeDoBooksFacade.shared.localization.setCustomLocalizations(overrides)
```

Events Example

```
private var cancellables: Set<AnyCancellable> = []

func observeEvents() {
    WeDoBooksFacade.shared.events.finishBookTapped
        .sink { isbn in
            print("Finish tapped for \(isbn)")
        }
        .store(in: &cancellables)
}
```

Cross References

- DocC: Configuration.md

- DocC: Styling.md
- DocC: Localization.md
- DocC: Images.md
- DocC: Events.md

Headless Audio and CarPlay

Goal

Use SDK audiobook runtime without presenting SDK player UI, and optionally integrate transport controls in host app surfaces.

Headless Flow

1. Ensure setup + sign-in is completed.
2. Load audiobook checkout with `headlessAudioPlayer.loadBook(...)`.
3. Use `play`, `pause`, `seek`, `setRate` controls.
4. Observe `onStateChange` and `onError` on main queue.

Headless Example

```
func startHeadless(_ checkout: Checkout) {
    do {
        try WeDoBooksFacade.shared
            .headlessAudioPlayer
            .loadBook(
                book: checkout,
                startPosition: 0
            )

        try WeDoBooksFacade.shared
            .headlessAudioPlayer
            .play()
    } catch {
        print("Headless failed: \(error)")
    }
}
```

CarPlay Example

```
func wireNowPlaying() {
    _ = WeDoBooksFacade.shared.carPlay.nowPlayingInfo
        .sink { info in
            print("Now playing info changed: \(String(describing: info))")
        }
}
```

Notes

- Headless and SDK Player UI share the same underlying runtime.
- Runtime ownership collisions are surfaced as deterministic errors.
- CarPlay controls are non-throwing helpers over the shared runtime.

Cross References

- DocC: HeadlessAudioPlayer.md
- DocC: CarPlay.md

Error Handling and Troubleshooting

Goal

Provide quick diagnosis paths for common setup/auth/open/storage/headless failures.

Error Families

- `SetupError`: setup lifecycle and reader initialization.
- `SignInError`: auth and token/factory integration issues.
- `CheckoutError` / `OpenCheckoutError`: checkout/open/presentation failures.
- `StorageOperationError`: offline and download issues.
- `HeadlessAudiobookError`: headless runtime state and arbitration errors.

Troubleshooting Matrix

Use this as a quick lookup when mapping SDK errors to host-app diagnostics:

- `setupAlreadyCalled`: setup executed more than once. First check app lifecycle and singleton setup location.
- `firebaseNotConfigured`: Firebase app config is missing or invalid. First check adapter factory and config plist path.
- `noUserSignedIn`: user-dependent API was called before sign-in. First check the auth token flow and sign-in result handling.
- `aBookIsAlreadyOpen`: runtime ownership collision. First check for existing SDK player/reader UI or headless playback.
- `downloadNotAllowedOnCellular`: cellular download policy blocked the request. First check `configuration.allowEbookDownloadUsingMobileData`.

Diagnostics Checklist

1. Confirm `setup(...)` succeeded this launch.
2. Confirm expected backend mode (`streaming` or `library`).
3. Confirm current user id is non-nil before user-scoped APIs.
4. Confirm runtime ownership before open/headless transitions.
5. Confirm network/download policy assumptions.

Cross References

- DocC: ErrorHandling.md